MenuSwipe

Dossier de lancement – M1 MIAGE Apprentissage

v0.1.0-SNAPSHOT – 20 octobre 2026

Équipe, comptes développeurs et dépôt

Membre	Compte GitHub	Email des commits
HAJAR JELTHI	@Hajar-JLT2	jelthihajar@gmail.com
RAYANE KACHBI	@Rayanekachbi	kachbirayane27@gmail.com

Dépôt : `https://github.com/Rayanekachbi/menuswipe`

Page de projet : statut et URL à confirmer avant dépôt.

Chaque membre vérifiera l'association de son email Git à son compte GitHub. La répartition est de bout en bout, avec relecture croisée des PR.

Projet et caractère innovant

Problème et public. MenuSwipe s'adresse aux étudiants et actifs souhaitant organiser leurs repas selon leurs goûts, leur budget et les ingrédients disponibles. Le besoin retenu est de coordonner les recettes de la semaine pour limiter les achats et les restes.

Proposition. L'utilisateur découvre des recettes sous forme de cartes qu'il apprécie ou rejette. L'application adapte les suggestions et calcule un menu sous contraintes. Un « j'aime » actualise les préférences ; seul l'ajout confirmé au menu réserve des ingrédients.

Entité pivot : le menu hebdomadaire. Ses recettes alimentent les calculs de quantités, substitutions, courses et préparation. Une recette pourra être recommandée parce qu'elle réutilise un ingrédient mobilisé par un autre repas.

Différenciation recherchée. Le projet associe découverte personnalisée et planification globale explicable. Il calcule des sélections, affectations et ordonnancements au-delà de la consultation de recettes. La comparaison avec les solutions existantes reste à réaliser ; aucune nouveauté absolue n'est revendiquée.

L'objectif est de réduire les restes estimés, sans garantir zéro gaspillage. Les prix, recettes et plans de magasin seront des données de démonstration, sans connexion commerciale.

Scénarios d'usage

1. **F1 – Découverte.** En tant qu'utilisateur, j'apprécie ou rejette des recettes afin d'obtenir un flux adapté à mes goûts et au menu.
2. **F2 – Composition.** En tant qu'étudiant, je renseigne mon budget, mon stock et le nombre de repas afin d'obtenir un menu limitant achats et restes.
3. **F3 – Substitution.** En tant qu'utilisateur, je demande un remplacement d'ingrédient afin d'adapter une recette à mes disponibilités.
4. **F4 – Préparation.** En tant qu'utilisateur, je renseigne mes équipements afin d'organiser plusieurs préparations sans conflit de ressources.
5. **F5 – Calendrier.** En tant qu'utilisateur, je renseigne les dates limites afin de placer les repas dans des créneaux compatibles.
6. **F6 – Courses.** En tant qu'utilisateur, je consulte une liste ordonnée selon un magasin de démonstration afin de réduire mes déplacements.

Plan des features

Chaque feature traverse les quatre couches : **BD, back, API REST et front**. Elle comprend une migration Flyway, un algorithme métier, des endpoints et un scénario démontrable.

Notre équipe est actuellement composée de deux personnes. Le plan prévoit donc **2 membres × 3 itérations = 6 fonctionnalités majeures**, soit une fonctionnalité par membre et par itération.

Chaque membre réalise ses fonctionnalités de bout en bout. La conception et les PR font l'objet d'une relecture croisée. Ce plan correspond à l'effectif actuel, sous réserve de la régularisation du groupe avec l'équipe enseignante.

Version	Membre A : HAJAR	Membre B : RAYANE
v0.1.0 24 novembre	**F1. Flux adaptatif de recettes** Algorithme : LinUCB.	**F2. Composition du menu sous budget** Algorithme : branch-and-bound.
v0.2.0 8 décembre	**F3. Substitution d'ingrédients** Approche : filtrage métier et classement par similarité cosinus.	**F4. Organisation du batch cooking** Algorithme : ordonnancement par liste sous contraintes de ressources.
v1.0.0 19 janvier	**F5. Calendrier des repas** Algorithme : flot à coût minimal.	**F6. Parcours de courses** Algorithmes : Dijkstra, plus proche voisin et amélioration 2-opt.

Périmètre et cohérence

- **F1 :** recommander selon les réactions, les recettes et le menu.
- **F2 :** choisir les repas sous budget en intégrant stock, paquets entiers et restes.
- **F3 :** filtrer les remplacements compatibles et classer leurs profils culinaires ; confirmation humaine.
- **F4 :** placer les opérations selon leurs dépendances et les équipements disponibles.
- **F5 :** affecter les repas aux créneaux admissibles, à dates et coûts de placement fixés.
- **F6 :** calculer puis améliorer le parcours de courses lié au menu.

Les fonctionnalités se rattachent au menu hebdomadaire. Les modules échangent par leurs ports. F3 devra dépasser un simple score de similarité : les contraintes de remplacement et leurs effets sur les quantités et le menu seront explicités dans sa fiche.

Limites et validation

Les choix sont prévisionnels. Chaque fiche précisera la complexité, les alternatives et les invariants. Les recherches combinatoires seront bornées ; les heuristiques ne garantissent pas l'optimalité. Les tests couvriront budget, stocks, exclusions et conflits de ressources, avec graines fixes et petites instances de référence.

Architecture

Front → API REST → cas d'utilisation et domaine → persistance → BD.

Couche	Choix	Remarques
Front	HTML, CSS, JavaScript natif	`front/`, client REST uniquement
API REST	Spring Boot 4 / MVC, OpenAPI	`/api/v1`, Swagger UI, erreurs RFC 9457
Back	Java 21, Spring Boot 4, JPA, Bean Validation	Architecture hexagonale
BD	PostgreSQL 17 ; H2 pour les tests	Flyway ; migration par feature
Qualité	JUnit 5, AssertJ, ArchUnit, JaCoCo, Codecov, SonarCloud	CI GitHub Actions
Conteneurs	Docker Compose	BD en v0.1.0 ; application en v0.2.0

Architecture hexagonale. Chaque module distingue `domain` en Java pur, `application` avec ports et services sans Spring, `adapter` pour REST/JPA et `config` pour l'assemblage. Les dépendances pointent vers le domaine ; les modules échangent par leurs ports. ArchUnit contrôle ces règles.

Modèle commun. Menu, repas, recettes, ingrédients et quantités constituent le socle. Stocks, conditionnements, prix indicatifs et unités normalisées permettent les calculs. Le coût porte sur les paquets entiers ; les restes sont évalués par ingrédient. Les dates sont renseignées, sans inférence de sécurité alimentaire.

Démonstration. Le profil `demo` charge quelques dizaines d'entrées via un script SQL répétable et idempotent dans `src/main/resources/db/demo/`.

Qualité du code. Clean Code, SOLID, objets valeur immuables, invariants à la construction et patterns justifiés : Strategy et Adapter lorsqu'ils répondent à un besoin. Les précautions OWASP comprennent validation, échappement, requêtes paramétrées et secrets hors dépôt. Le quality gate SonarCloud devra être vert.

Livraison. CI : build, tests, couverture et documentation. Tags `v*` : release du JAR et du PDF. `site/` : GitHub Pages. La compatibilité PostgreSQL sera vérifiée en complément de H2.

Démarche pour chaque feature

Les étapes sont suivies dans l'ordre, après validation de la précédente.

Étape	Ce qu'on fait	Terminé quand
1. Idéation	Besoin, algorithme, complexité, alternatives, impacts BD/API ; issue et fiche.	Fiche validée par un autre membre.
2. Back	Migration, domaine, ports, services, persistance et assemblage.	Compilation et ArchUnit réussis.
3. Tests	Cas limites, invariants, propriétés à graine fixe, faux adaptateurs et persistance.	`./gradlew build` passe ; chaque critère est couvert.
4. API REST	Endpoints, DTO, erreurs, OpenAPI et tests API.	Swagger UI disponible et tests verts.
5. Front	Écrans consommant uniquement l'API.	Scénario vérifié ; PR relue ; fiche et araignée complétées.

Protocole d'usage de l'IA

Principe : l'IA assiste, l'humain reste l'auteur.

Chaque étudiant doit comprendre, expliquer, modifier et refaire sans aide le cœur de ses fonctionnalités. Le cœur des algorithmes est écrit par les étudiants.

L'assistance progresse étape par étape : explication avant production, décision humaine et vérification de compréhension. Les règles retenues seront versionnées dans `CLAUDE.md`.

Activité	IA	Humain
Idéation et conception	Critique et propose des options	Choisit, justifie et valide la fiche
Algorithme	Explique et relit après une tentative	Conçoit et écrit le cœur
Code technique	Propose un premier jet	Relit chaque ligne, adapte et vérifie
Tests	Suggère des cas	Définit les propriétés et vérifie la détection des défauts
CI/CD et conteneurs	Propose workflows et Dockerfile	Comprend, exécute et gère les secrets
Documentation	Structure et reformule	Vérifie les faits et valide le contenu

Interdits

- Commiter du code non compris.
- Transmettre des secrets ou des données personnelles à l'IA.
- Commiter sous une autre identité.
- Présenter comme humain un travail produit par l'IA.

Traçabilité dans les PR

Chaque pull request contient une section « Usage de l'IA » : outil utilisé, objet des demandes, éléments repris, modifiés ou rejetés, et vérifications humaines réalisées.

Déclaration par compétence

Le fichier `doc/part-ia.json` alimente le graphe en araignée. Les plafonds proposés restent à valider par l'équipe.

Niveau	Compétences	Plafond proposé
Haut niveau	Idéation, conception, spécification, algorithmique, validation	30 % par axe
Bas niveau	Code, tests, CI/CD, rédaction, débogage	70 % par axe

Ces limites sont distinctes des contributions réelles. Les parts sont mesurées lorsque cela est pertinent, estimées et expliquées sinon. Tout dépassement est déclaré. Le plafond ne permet pas de déléguer le cœur algorithmique.

Usage pour ce dossier

ChatGPT a contribué aux ajustements de conception, aux extensions fonctionnelles et à la rédaction initiale. Sa contribution rédactionnelle est majoritaire. Les valeurs par compétence restent à évaluer après relecture humaine ; le respect des plafonds proposés n'est pas établi.

Documentation et rendu

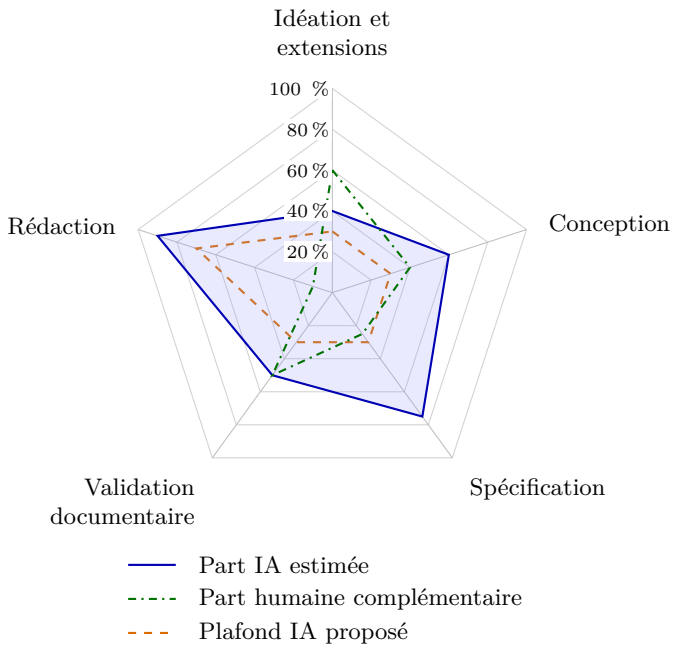
La source exigée est `doc/dossier.md`. La commande `make -C doc` produit `doc/dossier.pdf`. Le document est versionné et actualisé à chaque livraison. Chaque membre dépose individuellement le rendu collectif.

Annexe : part IA / part humaine

Périmètre : préparation du dossier R1.

L'idée initiale a été apportée par l'équipe. L'IA a participé à son approfondissement et à la rédaction du dossier.

Les valeurs représentées sont des propositions qualitatives de l'assistant, non mesurées et non validées. Elles ne constituent pas une déclaration finale de l'équipe. Les activités non réalisées ne sont pas assimilées à une contribution humaine de 100 %.



Suivi à compléter. Ce graphique provisoire ne couvre que cinq axes documentaires. Le suivi du cours doit aussi renseigner algorithmique, code, tests, CI/CD et débogage, en indiquant « non évalué » lorsque nécessaire. Les plafonds proposés ne sont pas des mesures ; les dépassements doivent être commentés après validation.